
Specifying Software Requirements

Project: Bocatto Valley
Revision



November 2025

Document file

Date	Revision	Author	Verified dep. quality.

Document validated by the parties on date:

By the customer	By the supply company
Mrs.	Mrs./Mrs.

Content

1 INTRODUCTION	4
1.1 Purpose	5
1.2 Scope	5
1.3 Staff involved	5
Ricardo Andres Lainez Suarez	5
1.4 Definitions, acronyms and abbreviations	6
1.5 References	6
1.6 Summary	6
2 OVERVIEW	7
2.1 Product Outlook	7
2.2 Product Functionality	7
2.2.1 Customer-Oriented Features	7
• Digital Ordering & Payment: Menu navigation, order customization (including allergens), cart management, offer application, and online payment processing	7
• Quick Service: Real-time order status tracking and use of QR codes for tableside orders.	7
• Reservations: Online availability and table reservation consultation, with reservation history and management.	7
• Interaction: Registration/Authentication, rating dishes and sending comments about the experience.	7
2.3 User characteristics	7
2.4 Restrictions	8
2.4.1 Customer-Oriented Features	8
Defined Stack: System development must strictly adhere to the following technologies and versions to limit the execution environment:	8
• Backend: Node.js runtime (v18+) and Express framework (v4.18+).	8
• Database: MongoDB Atlas (Cloud), managed by Mongoose (v8.0+).	8
• Frontend: HTML5, CSS3, y JavaScript (ES6+).	8
Security Standards: User authentication management must be implemented using JSON Web Tokens (JWT). In addition, password hashing should be done with bcryptjs.	8
Development Environments: Local development environments must operate on the following endpoints: Frontend (http://localhost:5500 or Live Server) and Backend API (http://localhost:3000).	8
2.5 Assumptions and dependencies	8
3 SPECIFIC REQUIREMENTS	9
3.1 Common interface requirements	14
3.1.1 User Interfaces	14
3.1.2 Hardware Interfaces	15
3.1.3 Software interfaces	15
3.1.4 Communication Interfaces	16
3.2 Functional requirements	16

3.2.1 RF1 – User Authentication & Role Control	16
3.2.2 RF2 – Digital Menu Catalog	16
3.2.3 RF3 – Meal Customization & Allergy Management	17
3.2.4 RF4 – Shopping Cart Management	18
3.2.5 RF5 – Payment Processing & Ticket Validation	18
3.2.6 RF6 – Reservation System	19
3.2.7 RF7 – Reviews & Ratings	20
3.2.8 RF8 – Promotions & Loyalty Program	20
3.2.9 RF9 – Administrative Dashboard	21
3.2.10 RF10 – Error Handling & Recovery	21
3.3 Non-functional requirements	22
3.3.1 Performance requirements	22
3.3.2 Safety	23
3.3.3 Usability	23
3.3.4 Security	24
3.3.5 Scalability	24
3.3.6 Reliability	24
3.3.7 Availability	25
3.3.8 Maintainability	25
3.3.9 Portability	25
4 APPENDICES	26
4.1 Database model	26
4.2 Project UML Diagrams	26
4.2.1 Class Diagram	26
4.2.2 Use Case Diagram	27
4.3 Project Prototypes	28
5 TEST PLAN	28

1 Introduction

The introduction of this Software Requirements Specification (SRS) sets out the overall context of the project and the structure of the document. It provides a high-level view of the Bocatto Valley system, defining the purpose of this documentation, the functional scope of the product to be developed, and the primary audience it is aimed at.

1.1 Purpose

The primary purpose of this document is to define, clearly and unambiguously, all functional and non-functional requirements of the Bocatto Valley system. This SRS will serve as the formal and contractual basis for the design, development, testing, and acceptance of the final product, ensuring that the system meets the expectations of the business and users.

- **Development Team:** For the coding, integration and implementation of the system, ensuring that the technical specifications are met.
- **Project Management:** For scope control, risk management, and development schedule oversight.
- **Customer:** For review, validation and final approval that the system meets the operational and business needs of the fast food restaurant.

1.2 Scope

The product to be developed is an Online Order Management and Operations System for fast food restaurants, called Bocatto Valley. This system is designed to handle the high order concurrency and need for fast service inherent in this type of business.

1.3 Staff involved

Name	Ricardo Andres Lainez Suarez
Role	Analyst, designer and programmer
Professional category	Software Engineering
Responsibilities	Design the system model to be followed, code, analyze system development, and perform tasks specified in the project schedule.
Contact Information	ralainez@espe.edu.ec

Name	Llumiquinga Ñacato Ariel José
Role	Analyst, designer and programmer
Professional category	Software Engineering
Responsibilities	Design the system model to be followed, code, analyze system development, and perform tasks specified in the project schedule.
Contact Information	aillumiquinga2@espe.edu.ec

Name	Santi Cruz Jeancarlo Javier
Role	Analyst, designer and programmer
Professional category	Software Engineering

Responsibilities	Design the system model to be followed, code, analyze system development, and perform tasks specified in the project schedule.
Contact Information	jjisanti@espe.edu.ec

1.4 Definitions, acronyms and abbreviations

Acronym	Definition
SRS	Software Requirements Specification
Bocatto Valley	Formal name of the online ordering and booking system that is the subject of this project.
CRUD	Create, Read, Update and Delete. The four basic functions of data persistence.
Dashboard	User interface for the Administrator, designed for system management and monitoring.
QR	Two-dimensional barcode used to access the menu or initiate an order at the table.
API	Application Programming Interface. A set of rules and protocols used to build and interact with the system <i>backend</i> .
Front-end	Part of the system with which the user interacts directly.
Back-end	Part of the system that processes business logic, manages the database and security.

1.5 References

Reference	Title	Route	Date	Author
[IEEE-830-1998]	IEEE Recommended Practice for Software Requirements Specifications	International Standard	1998	The Institute of Electrical and Electronics Engineers (IEEE)
[DCU-BV]	Bocatto Valley System Use Case Diagram	Internal Project Documentation	November 2025	Bocatto Valley Project Team
[FD-BV]	Bocatto Valley Project Features and Scope Definition	Internal Project Documentation	October 2025	Bocatto Valley Project Team

1.6 Summary

This document specifies the requirements for the development of the Bocatto Valley advanced web system, designed for a fast food restaurant. The system will allow customers to order and customize food online, pay electronically and make table reservations, while also offering real-time order tracking. Both detailed and

non-functional requirements are included, as well as an overview of the system, its interfaces (Client and Administrator), the users involved, and operational constraints. This specification will serve as the formal basis for system design, implementation, testing, and validation, ensuring compliance with the more than nine advanced operations outside of the CRUD standard.

2 Overview

2.1 Product Outlook

Bocatto Valley is an Advanced Web System developed as a standalone product that serves as the central digital platform for the operation of a fast food restaurant. The system is aimed at making it easier for customers to order and personalise food, make electronic payments and reserve tables, while also offering real-time order tracking. For its implementation, it uses a Node.js/Express-based architecture for the backend and MongoDB Atlas for data warehousing. The frontend is developed with HTML5, CSS3, and JavaScript (ES6+), securing the user interface. Security and authentication are managed using bcryptjs and JSON Web Tokens (JWT).

2.2 Product Functionality

This section presents a summary of the main functionalities that the Bocatto Valley system should offer. The functionalities are organized in logical blocks aimed at the end user (*Customer*) and the management staff (*Administrator*), ensuring that the scope is perfectly understandable for any interlocutor.

Customer-Oriented Features

2.2.1 Customer-Oriented Features

- **Digital Ordering & Payment:** Menu navigation, order customization (including allergens), cart management, offer application, and online payment processing
- **Quick Service:** Real-time order status tracking and use of QR codes for tableside orders.
- **Reservations:** Online availability and table reservation consultation, with reservation history and management.
- **Interaction:** Registration/Authentication, rating dishes and sending comments about the experience.

2.2.2 Administrator-Oriented Features

- **Operational Management:** Total control (CRUD) of the menu, prices, offers and promotions.
- **Order Flow:** Reception and management of the life cycle of all active orders and reservations, facilitating shipment to the kitchen.
- **Control and Reporting: Moderation of customer comments** and ratings. Management of user profiles (including the creation of other administrators). Generation of sales and operation reports.

2.3 User characteristics

User Type	Children until adults
Formation	
Skills	medium
Activities	Use the internet

2.4 Restrictions

The following are the mandatory limitations imposed on design, technologies, and operation that must be adhered to during the development of the Bocatto Valley system.

2.4.1 Customer-Oriented Features

Defined Stack: System development must strictly adhere to the following technologies and versions to limit the execution environment:

- Backend: Node.js runtime (v18+) and Express *framework* (v4.18+).
- Database: MongoDB Atlas (Cloud), managed by Mongoose (v8.0+).
- Frontend: HTML5, CSS3, y JavaScript (ES6+).

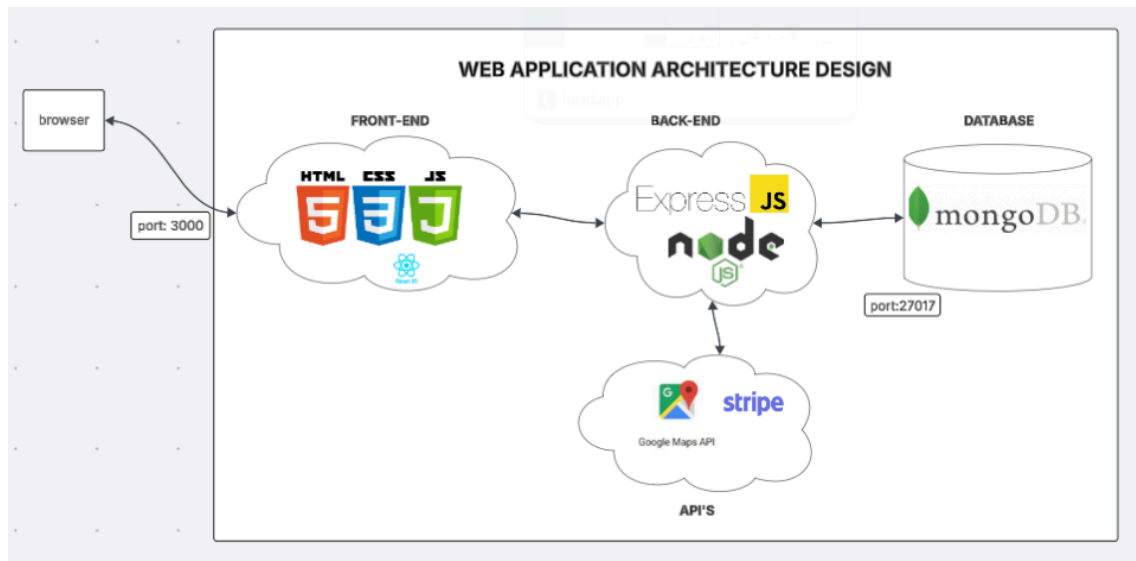
Security Standards: User authentication management must be implemented using JSON Web Tokens (JWT). In addition, *password hashing* should be done with bcryptjs.

Development Environments: Local development environments must operate on the following *endpoints*: Frontend (<http://localhost:5500> or *Live Server*) and Backend API (<http://localhost:3000>).

2.5 Assumptions and dependencies

- This section details the external factors that are assumed to be true and that, if they change, would directly affect the system requirements.
- **Payment Gateway Dependency:** The project depends on the provision of API credentials and documentation from an external Payment Gateway for the Online Payments (UC16) functionality to be implemented.
- **Hosting Support for Architecture:** It is assumed that the *hosting environment* in production will support the *Node.js runtime* and WebSockets technology, necessary for Real-Time Tracking (UC8).
- **Transactional Integrity:** It is assumed that Reserve (UC11) and Payment (UC16) should be modeled as an atomic transaction to ensure the integrity of the table data.
- **Availability of Initial Data:** It is dependent on the restaurant to provide all the initial content (menu, prices, images, table configuration) in a timely manner to begin the testing and deployment phase.

2.6 Architecture prototype



3 Specific requirements

The following functional and non-functional requirements define the expected behavior of the system and the conditions under which it must operate.

RF 1 – User Authentication and Access Control

Field	Description
Requirement Number	RF 1
Requirement Name	User Authentication & Role Management
Guy	Requirement
Requirement Source	Project Backlog (F1)
Requirement Priority	High / Essential

Description:

The system must allow users to register, log in, and log out securely. The platform should support two roles: **Client** and **Administrator**, each with different permissions.

Sub-requirements:

- RF 1.1 – Registration requires unique email and encrypted password.
- RF 1.2 – Administrators can manage users and assign roles.

RF 2 – Digital Menu Catalog

Field	Description
Requirement Number	RF 2
Requirement Name	Menu and Product Management
Guy	Requirement
Requirement Source	Project Backlog (F2)
Requirement Priority	High / Essential

Description:

The system must display a **digital menu** of food and beverages with images, prices, and product descriptions. Administrators can perform CRUD operations on products and categories.

Sub-requirements:

- RF 2.1 – Menu items organized by category (e.g., burgers, desserts, drinks).
- RF 2.2 – Each product includes image, description, price, and ingredients.
- RF 2.3 – Administrators can add, edit, or delete products.

RF 3 – Meal Customization & Allergies

Field	Description
Requirement Number	RF 3
Requirement Name	Customization and Allergy Preferences
Guy	Requirement
Requirement Source	Problem & Overview
Requirement Priority	Medium / Deseado

Description:

Users must be able to customize their meals by removing allergens or adding ingredients (e.g., sauces, cheese). Prices must recalculate automatically based on selected options.

Sub-requirements:

- RF 3.1 – Ingredients are linked to each product.
- RF 3.2 – Users can save allergy preferences in their profile.
- RF 3.3 – The system displays warnings for allergenic ingredients.

RF 4 – Cart and Checkout

Field	Description
Requirement Number	RF 4
Requirement Name	Cart Management and Checkout
Guy	Requirement
Requirement Source	Project Backlog (F3)
Requirement Priority	High / Essential

Description:

The system must allow users to add, modify, and remove items from their shopping cart. The cart must display item count, subtotal, taxes, and final total in real time.

Sub-requirements:

- RF 4.1 – Quantities can be modified directly in the cart view.
- RF 4.2 – Taxes and discounts are recalculated dynamically.
- RF 4.3 – Empty cart triggers a “no items” message.

RF 5 – Online Payment Integration

Field	Description
Requirement Number	RF 5
Requirement Name	Payment Gateway and Transaction Management
Guy	Requirement
Requirement Source	Problem & Feature Plan
Requirement Priority	High / Essential

Description:

Integrate a secure payment gateway (Stripe / MercadoPago) for online orders and reservations. Payment confirmation updates order status to “Approved”.

Sub-requirements:

- RF 5.1 – Payments must be encrypted and processed over HTTPS.
- RF 5.2 – Failed transactions revert the order to “Pending”.
- RF 5.3 – A digital receipt must be generated automatically.

RF 6 – Reservation System

Field	Description
Requirement Number	RF 6
Requirement Name	Table Reservation and Management
Guy	Requirement
Requirement Source	Project Analysis
Requirement Priority	High / Essential

Description:

The system must allow customers to reserve tables by selecting date, time, and number of guests. The admin panel displays upcoming reservations.

Sub-requirements:

- RF 6.1 – Time slots cannot overlap for the same table.
- RF 6.2 – Confirmation email sent to user after successful reservation.
- RF 6.3 – Admins can cancel or reassign reservations.

RF 7 – Customer Reviews and Ratings

Field	Description
Requirement Number	RF 7
Requirement Name	Feedback and Rating Module
Guy	Requirement
Requirement Source	Feature Plan
Requirement Priority	Medium / Deseado

Description:

Users can leave comments and rate products from 1 to 5 stars. Administrators can moderate, approve, or delete reviews.

Sub-requirements:

- RF 7.1 – A client may post only one rating per product.
- RF 7.2 – Ratings are averaged and displayed under each product.
- RF 7.3 – Admins can view pending comments for approval.

RF 8 – Promotions and Loyalty Program

Field	Description
Requirement Number	RF 8
Requirement Name	Promotional Offers and Loyalty Rewards
Guy	Requirement
Requirement Source	Competitor Analysis
Requirement Priority	Medium / Deseado

Description:

The system must allow users to apply coupons or loyalty points at checkout to obtain discounts.

Sub-requirements:

- RF 8.1 – Coupons are validated before payment.
- RF 8.2 – Loyalty points accumulate after each purchase.
- RF 8.3 – Admins can create and manage promotional offers.

RF 9 – Administrative Dashboard

Field	Description
Requirement Number	RF 9
Requirement Name	Dashboard and Analytics
Guy	Requirement
Requirement Source	Feature Plan
Requirement Priority	High / Essential

Description:

The system must include an administrator dashboard to view sales reports, inventory, and customer activity.

Sub-requirements:

- RF 9.1 – Show sales by day, week, and month.
- RF 9.2 – Display low-stock alerts automatically.
- RF 9.3 – Allow CSV or PDF export of reports.

RNF 1 – System Performance and Security

Field	Description
Requirement Number	RNF 1
Requirement Name	Performance, Security, and Availability
Guy	Restriction
Requirement Source	Quality Attributes
Requirement Priority	High / Essential

Description:

The web application must respond to user actions in less than **2 seconds** under standard load. Sensitive data (passwords, payments) must be **encrypted**. The system should maintain **99% uptime** and be hosted in a secure environment (SSL/TLS 1.2+).

3.1 Common interface requirements

3.1.1 User Interfaces

The Bocatto Valley Web Application is a responsive and intuitive platform designed to provide a smooth user experience for both customers and administrators. The interface must follow a consistent layout, color palette, and navigation structure across all devices.

General Design Principles

- The user interface must be **responsive**, adapting to mobile, tablet, and desktop resolutions.
- The design follows a **modern and minimalist style**, prioritizing readability and accessibility.
- **Color scheme:** shades of red, white, and light gray (aligned with the Bocatto Valley brand).
- **Typography:** Sans-serif fonts (e.g., Open Sans, Poppins, or Roboto) for readability.
- **Accessibility:** All UI elements should comply with WCAG 2.1 standards.

3.1.2 Hardware Interfaces

The system is designed to be **fully web-based**, accessible via standard web browsers on both desktop and mobile devices. No dedicated hardware is required beyond the hosting infrastructure.

Component	Description
Client Devices	Any modern device (PC, laptop, tablet, smartphone) capable of running a web browser (Chrome, Edge, Safari, Firefox).
Server Environment	Cloud-based hosting (e.g., AWS, Azure, or similar) supporting Node.js runtime environment.
Network	Requires stable internet connection (minimum 2 Mbps for smooth operation).
Peripheral Devices	Optional printers for order receipts in administrative environments.

3.1.3 Software interfaces

The system relies on several internal and external software components that work together to deliver full functionality.

Interface	Description	Format / Protocol
Frontend	Developed with html, css, js / Next.js , consuming RESTful APIs.	HTTP(S) requests using JSON payloads.
Backend	Built on Node.js + Express.js , managing authentication, business logic, and database operations.	REST API, JSON data structure.
Database (MongoDB Atlas)	Centralized cloud NoSQL database for products, users, orders, and reservations. Collections include: users, products, orders, reservations, reviews, payments, offers, categories.	CRUD operations using Mongoose ODM, JSON documents.
Payment Gateway (Stripe / MercadoPago)	Handles secure online payments and validations.	HTTPS API: POST /api/payments with encrypted tokens.
Email Service (SMTP / Nodemailer)	Sends confirmation emails for registration, orders, and reservations.	SMTP protocol, JSON message templates.
Authentication (JWT)	Provides secure token-based user sessions.	JWT tokens in headers (Authorization: Bearer).

Integration Requirements:

- The system must log all external API interactions (success/failure).
- API keys must be stored securely in environment variables (.env).
- Third-party services must support HTTPS and use valid SSL certificates.

3.1.4 Communication Interfaces

- HTTPS for all transactions.
- JSON format for client-server data exchange.
- RESTful API structure for modular communication.

3.2 Functional requirements

3.2.1 RF1 – User Authentication & Role Control

Aspect	Specification
Ticket Validity Check	Validates login credentials (email/password). Ensures user exists and account is active before granting access.
Exact Sequence of Operations	1. User enters credentials. 2. System verifies in DB. 3. If valid → issue JWT token. 4. Assign user role (<i>Client</i> , <i>Admin</i>).
Response to Abnormal Situations	Invalid credentials → error 401; inactive user → blocked; system error → 500 logged.
Parameters	Email, password, role.
Output Generation	JWT token + user profile (name, role, permissions).
Input–Output Relationship	Input {email, password} → Output {token, userId, role}.
Database Logical Requirements	Collection users{_id, email, pass_hash, role, status}; unique index on email.

Description:

Handles secure authentication and access control for all users. Ensures that only valid sessions operate in the platform and that user roles define permitted actions.

3.2.2 RF2 – Digital Menu Catalog

Aspect	Specification
Ticket Validity Check	Confirms the request comes from an active session or public route.
Exact Sequence of Operations	1. User requests catalog. 2. System retrieves products. 3. Filters by category or tag. 4. Sends response.
Response to Abnormal Situations	Product not found → 404; DB timeout → 500.
Parameters	Category, tag, price range.
Output Generation	Product list with image, name, description, and price.
Input–Output Relationship	Input: GET /products?category=burgers → Output: [{name, price, image}].
Database Logical Requirements	Collection products{ _id, name, desc, price, imageUrl, category, tags[] }.

Description:

Displays the entire menu in a visually appealing, organized way, allowing users to browse, search, and filter products quickly.

3.2.3 RF3 – Meal Customization & Allergy Management

Aspect	Specification
Ticket Validity Check	Validates ingredient options and allergy list before applying changes.
Exact Sequence of Operations	1. User selects product. 2. Chooses to add/remove ingredients. 3. System recalculates price and warns of allergens.
Response to Abnormal Situations	Invalid ingredient ID → 422; allergy conflict → warning alert.
Parameters	ProductId, ingredient list, allergy list.
Output Generation	Custom meal object {product, modifications, finalPrice}.
Input–Output Relationship	Input {productId, remove:["onion"]} → Output {price: 8.50, warnings:["contains dairy"]}.

Database Logical Requirements	Collections ingredients, userAllergies, productIngredients.
-------------------------------	---

Description:

Allows personalized meal options based on dietary preferences or allergies. Ensures safe customization and correct price calculation.

3.2.4 RF4 – Shopping Cart Management

Aspect	Specification
Ticket Validity Check	Validates that each item in cart exists and stock is available.
Exact Sequence of Operations	1. Add item. 2. Update quantity/price. 3. Recalculate totals. 4. Persist cart in DB.
Response to Abnormal Situations	Invalid quantity → 400; stock unavailable → 409.
Parameters	ProductId, quantity, userId.
Output Generation	Cart summary {items[], subtotal, tax, total}.
Input–Output Relationship	Input {productId, qty:2} → Output {total:25.00}.
Database Logical Requirements	Collection carts{_id, userId, items[{productId, qty, price}], total}.

Description:

Manages users' shopping carts dynamically, maintaining data consistency and real-time price updates throughout the purchase flow.

3.2.5 RF5 – Payment Processing & Ticket Validation

Aspect	Specification
Ticket Validity Check	Confirms cart and totals before initiating payment.
Exact Sequence of Operations	1. Validate cart and total. 2. Create payment intent. 3. Process transaction. 4. Confirm via webhook.

Response to Abnormal Situations	Declined card → mark “Failed”; missing confirmation → rollback; timeout → retry.
Parameters	OrderId, amount, paymentMethod.
Output Generation	Receipt {orderId, total, status}.
Input–Output Relationship	Input {orderId, amount} → Output {status:'Approved', receiptUrl}.
Database Logical Requirements	Collections payments, orders with references by orderId.

Description:

Handles online payment securely, validating transaction data and issuing digital receipts to ensure accurate order tracking.

3.2.6 RF6 – Reservation System

Aspect	Specification
Ticket Validity Check	Ensures selected date/time/table is available.
Exact Sequence of Operations	1. User chooses date/time. 2. System checks availability. 3. Confirms reservation. 4. Sends confirmation email.
Response to Abnormal Situations	Overlapping reservation → 409; invalid date → 422.
Parameters	Date, time, pax.
Output Generation	Reservation confirmation {reservationId, status}.
Input–Output Relationship	Input {date, time, pax:4} → Output {status:'Confirmed'}.
Database Logical Requirements	Collections reservations, tables, availability.

Description:

Provides online table booking to improve customer service and reduce manual scheduling errors.

3.2.7 RF7 – Reviews & Ratings

Aspect	Specification
Ticket Validity Check	Ensures the user has purchased the product before reviewing.
Exact Sequence of Operations	1. Submit review. 2. Validate one-per-user rule. 3. Mark as pending. 4. Admin approval.
Response to Abnormal Situations	Duplicate review → 409; inappropriate text → 422.
Parameters	ProductId, rating, comment.
Output Generation	Review confirmation {reviewId, status:'Pending'}.
Input–Output Relationship	Input {rating:5} → Output {averageRating:4.8}.
Database Logical Requirements	Collection reviews{userId, productId, rating, comment, status}.

Description:

Allows users to express feedback, improving service transparency while maintaining moderation and quality control.

3.2.8 RF8 – Promotions & Loyalty Program

Aspect	Specification
Ticket Validity Check	Validates coupon code and user eligibility.
Exact Sequence of Operations	1. User inputs code. 2. Validate expiration and usage. 3. Apply discount. 4. Update loyalty points.
Response to Abnormal Situations	Expired coupon → 410; invalid code → 404.
Parameters	CouponCode, userId.
Output Generation	Discount applied and updated totals.
Input–Output Relationship	Input {coupon:'BOCATTO10'} → Output {discount:10%, total:27.00}.

Database Logical Requirements	Collections offers, loyalty.
--------------------------------------	------------------------------

Description:

Increases customer retention by rewarding purchases and allowing discounts through coupons and loyalty points.

3.2.9 RF9 – Administrative Dashboard

Aspect	Specification
Ticket Validity Check	Verifies user role = Administrator before access.
Exact Sequence of Operations	1. Admin logs in. 2. Requests metrics. 3. Data aggregated from DB. 4. Report generated.
Response to Abnormal Situations	Unauthorized → 403; empty data → 204.
Parameters	Date range, report type.
Output Generation	KPIs, charts, downloadable CSV/PDF.
Input–Output Relationship	Input GET /admin/metrics → Output {sales, stockAlerts}.
Database Logical Requirements	Collections orders, products, reports.

Description:

Gives administrators tools to monitor performance, manage stock, and generate analytical reports for decision-making.

3.2.10 RF10 – Error Handling & Recovery

Aspect	Specification
Ticket Validity Check	Validates process state before retry or rollback.
Exact Sequence of Operations	1. Detect error. 2. Log details. 3. Retry operation (max 3). 4. Notify user.

Response to Abnormal Situations	Handles timeouts, communication loss, and overflows safely.
Parameters	ErrorType, traceId, timestamp.
Output Generation	Standardized JSON error message.
Input–Output Relationship	Input: failed request → Output: {code, message, detail}.
Database Logical Requirements	Collection logs{_id, message, module, level, timestamp}.

Description:

Ensures the system can handle unexpected conditions gracefully, log errors, and recover operations without data loss.

3.3 Non-functional requirements

3.3.1 Performance requirements

Aspect	Specification
Load Capacity	The system must support 300 concurrent users and 50 requests per second on average. Peak tolerance up to 600 concurrent users for 5 minutes.
Response Time	95% of operations (login, menu, add-to-cart, checkout) must complete in less than 1.5 seconds . Reports must generate in under 10 seconds .
Throughput	The database and API should handle ≥ 15 writes/sec and ≥ 50 reads/sec under normal load.
Performance Monitoring	Metrics collected via logs and monitoring tools to detect latency, CPU usage, and slow queries.

Description:

Guarantees that Bocatto Valley runs efficiently under typical and peak loads, ensuring fast and responsive interactions for both customers and administrators.

Versión	1.0
Identification of the requirement	RNF-001

Name of the requirement	Performance
Characteristics:	Fast response, High concurrency
Description	The system must complete most critical actions (Orders, Payments) in less than 1.5 seconds. It must support 100 concurrent users without slowing down.
Prioridad	High

3.3.2 Safety

<i>Versión</i>	1.0
Identification of the requirement	RNF-002
Name of the requirement	security
Characteristics:	Secure
Description	The data stored must be secure and passwords must be encrypted.
Prioridad	High

3.3.3 Usability

<i>Versión</i>	1.0
Identification of the requirement	RNF-003
Name of the requirement	Usability
Characteristics:	Clean interface, Simple navigation
Description	The ordering interface must be easy for any non-technical customer and allow them to complete an order in a maximum of 5 clicks to speed up the purchase.
Prioridad	High

<i>Versión</i>	1.0
Identification of the requirement	RNF-004
Name of the requirement	Compatibility
Characteristics:	Works on modern browsers

Description	The system must be accessible from all current major browsers.
Prioridad	Medium

3.3.4 Security

Versión	1.0
Identification of the requirement	RNF-005
Name of the requirement	Security
Characteristics:	Data protection, Role-based access
Description	Passwords must be stored securely (encrypted). Only the Administrator role must be granted access to the internal restaurant management tools.
Prioridad	High

3.3.5 Scalability

Versión	1.0
Identification of the requirement	RNF-006
Name of the requirement	Scalability
Characteristics:	Support for traffic peaks
Description	The architecture must allow the system to easily increase its capacity to handle at least 30% more users and orders during peak hours without requiring a full redesign.
Prioridad	Medium

3.3.6 Reliability

Versión	1.0
Identification of the requirement	RNF-007
Name of the requirement	Reliability
Characteristics:	No lost bookings or money
Description	The system must be completely trustworthy when handling payments and reservations. If a payment fails, the table must not be booked (or vice versa). The system must

Versión	1.0
	correct any payment/booking error within 5 minutes to avoid double bookings or order confusion.
Prioridad	Medium

3.3.7 Availability

Versión	1.0
Identification of the requirement	RNF-008
Name of the requirement	Availability
Characteristics:	Constant service access
Description	The system must guarantee a minimum uptime of 99.5% during scheduled operating hours annually, excluding planned maintenance.
Prioridad	High

3.3.8 Maintainability

Versión	1.0
Identification of the requirement	RNF-009
Name of the requirement	Maintainability
Characteristics:	Code organized by parts
Description	The code must be organized into separate logical parts (modules, e.g., Orders and Reservations) so that fixing or updating one part does not damage others.
Prioridad	Medium

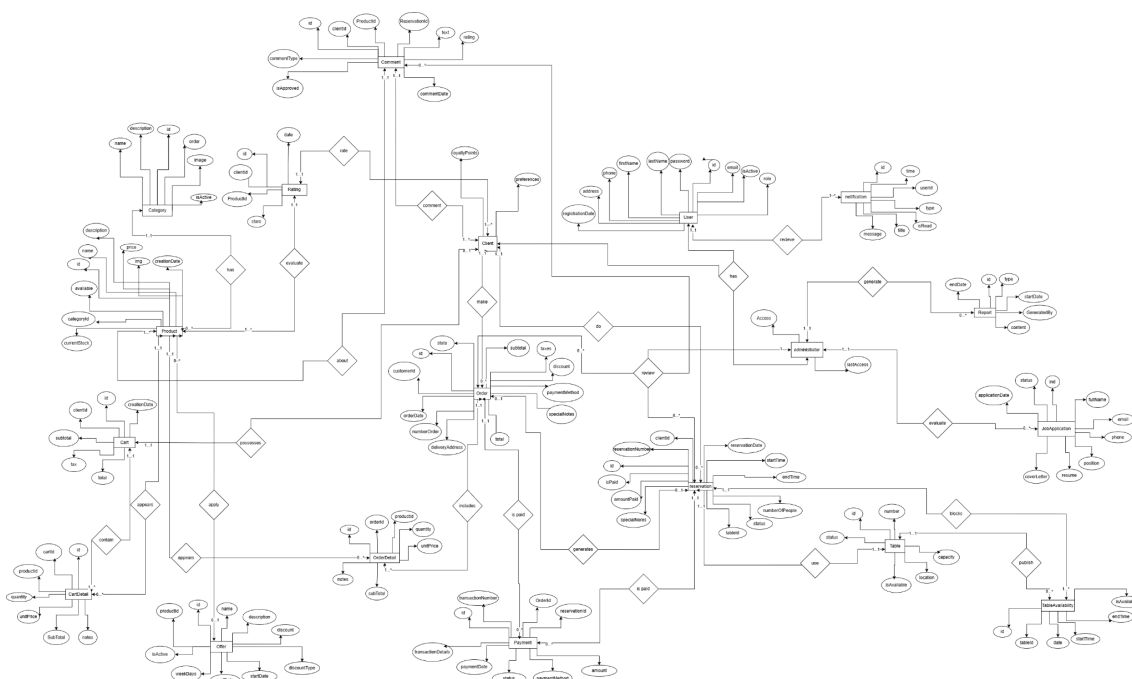
3.3.9 Portability

Versión	1.0
Identification of the requirement	RNF-010
Name of the requirement	Portability
Characteristics:	Mobile-friendly design

Versión	1.0
Description	The design must be fully responsive so that it looks and functions correctly on both mobile phones and desktop computers.
Prioridad	High

4 Appendices

4.1 Database model



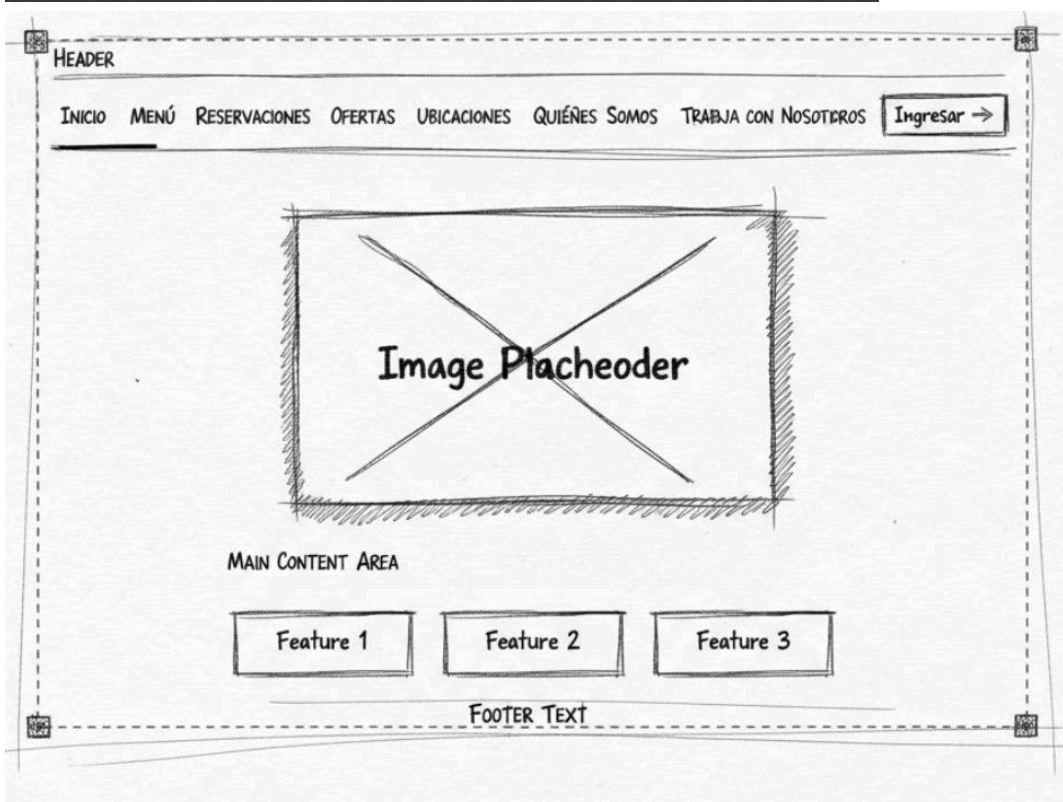
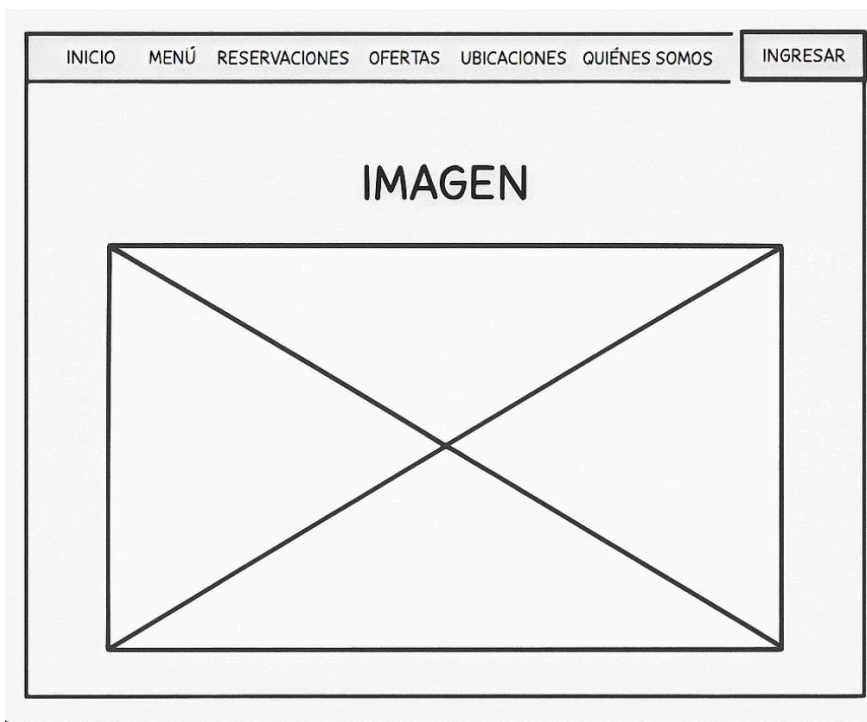
4.2 Project UML Diagrams

4.2.1 Class Diagram



4.3 Project Prototypes

Homepage



Restaurant Menu

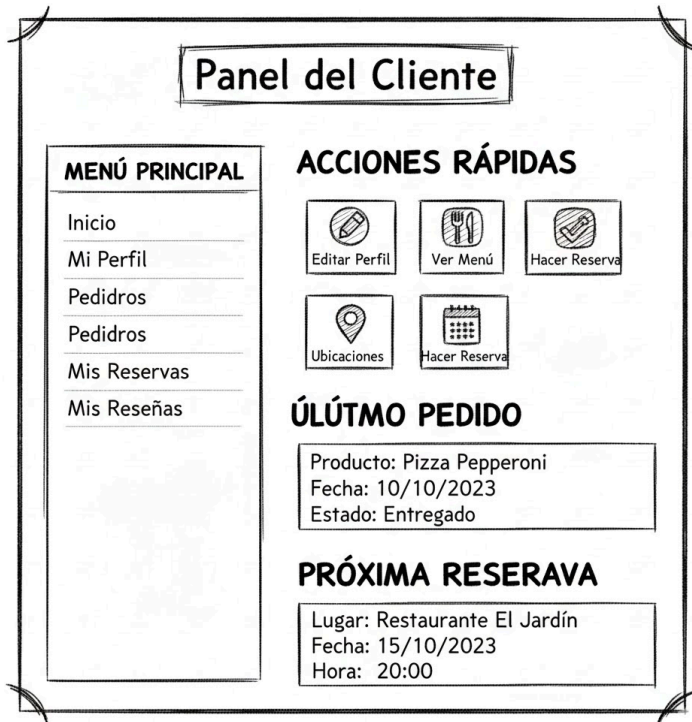


Reservations



Offers**Location**

Client dashboard



Who are we?



contact us

Contáctenos

Nombres

Apeilldos

Email

Telépono

Quiere conocer nuestras instalaciones
Sí ☐ ☐ No

Mensaje

Work with us?

Forma parte de nuestro equipo

Nombres

Apellidos

Cedula

Teléfono

Correo Electrónico

Área de Interés

¿Porqué Bocatto Valley?

Somos auta empresa líder
con un ambiente de trabajo
colaborativo.

Proceso Rápido

Revisamos su solicitud
en 48 horas.

5 Schedule

- **Total duration:** 16 weeks
- **Project start:** October 6, 2025
- **Methodology:** Hybrid approach – **Waterfall** (phases) + **SCRUM** (sprints, backlog, reviews).

Detailed Schedule – Week by Week

Phase 1 – Initiation, Analysis & Design (Weeks 1–4)

Week 1 – Oct 6, 2025 to Oct 12, 2025

Focus: Project definition and context

Activities:

- Search and select a suitable project proposal.
- Identify and confirm the **client**.
- Decide the **team name** and **project name**.
- Get general approval of the topic from client/instructor.

Deliverables:

- Short initial proposal document.
- Final team name and project name.
- Record of topic approval.

Week 2 – Oct 13, 2025 to Oct 19, 2025

Focus: Problem statement and initial requirements

Activities:

- Write the **problem proposal / problem statement**.
- Create the **system overview**.
- Develop the **background** (context, justification).
- Do a **comparison/competitor analysis**.
- Perform **requirements elicitation**:

- Functional requirements RF1–RF9.
- Non-functional requirement RNF1.

Deliverables:

- “Problem & Overview” document.
- Initial requirements specification (list of RFs and RNFs).

Week 3 – Oct 20, 2025 to Oct 26, 2025

Focus: System modeling

Activities:

- Create **class diagrams**.
- Create **use case diagrams** for RF1–RF9.
- Design the **ER model** for the database.
- Define how **SCRUM + Waterfall** will be combined in the project.
- Design **UI mockups** for main views:
 - Login, menu, cart, reservations, admin dashboard, etc.

Deliverables:

- UML package (use cases, class diagrams).
- ER diagram.
- Mockups/prototype of main screens.
- Document explaining the hybrid methodology (SCRUM + Waterfall).

Week 4 – Oct 27, 2025 to Nov 2, 2025

Focus: System architecture & first non-fully functional prototype

Activities:

- Define the **system architecture** (layers, technologies, patterns).
- Define responsibilities of **frontend**, **backend**, and **database** according to requirements.

- First implementation of functionalities:
 - **RF1** – User authentication & role management.
 - **RF2** – Digital menu catalog (basic).
 - **RF6** – Reservation system (initial version).
 - **RF9** – Admin dashboard (basic structure).
- Implement **login**.
- Build a **non-fully functional prototype** (navigable but incomplete).

Note: Some sub-requirements remain incomplete or need refinement, e.g. RF2.3, RF6.1, RF6.2, RF6.3, and additional details in other RFs.

Deliverables:

- Architecture document.
- Repository with a navigable (but not final) prototype.

Phase 2 – Agile Planning & Prototype Review (Weeks 5–6)

Week 5 – Nov 3, 2025 to Nov 9, 2025

Focus: Backlog, sprints, and requirements refinement

Activities:

- Build the **Product Backlog** with user stories for RF1–RF9 and RNF1.
- Define **sprints** (duration, goals, scope of each sprint).
- Refine and detail all requirements:
 - Sub-requirements such as RF2.1–2.3, RF6.1–6.3, etc.
- Prioritize requirements:
 - **High / Essential:** RF1, RF2, RF4, RF5, RF6, RF9, RNF1.
 - **Medium / Desired:** RF3, RF7, RF8.

Deliverables:

- Prioritized Product Backlog.
- Sprint plan for the remaining weeks.
- Updated and detailed requirements specification.

Week 6 – Nov 10, 2025 to Nov 16, 2025

Focus: Group inspections & prototype verification

Activities:

- **Cross-inspection** of projects with other teams.
- Verify current prototype functionalities (smoke testing).
- Fix critical issues in:
 - RF1 (authentication),
 - RF2 (menu),
 - RF6 (reservations),
 - RF9 (dashboard).
- Update the Product Backlog with new findings and feedback.

Deliverables:

- Inspection/review report.
- List of issues and improvements with priorities.
- Updated backlog.

Phase 3 – Development by Sprints (Weeks 7–14)

From here, the work is structured mainly in **2-week sprints**.

Sprint 1 – Core Features I (Weeks 7–8)

Week 7 – Nov 17, 2025 to Nov 23, 2025

Focus: Finalizing authentication and menu structure

Activities:

- Complete **RF1 – User Authentication & Role Management**:
 - **RF1.1** – Registration with unique email and encrypted password.
 - **RF1.2** – Admin can manage users and assign roles.

- Improve basic security (part of **RNF1**: password hashing, secure sessions, basic HTTPS configuration if possible).
- Refine **RF2 – Digital Menu Catalog**:
 - **RF2.1** – Menu items organized by category.
 - **RF2.2** – Each product with image, description, price, ingredients.
- Stabilize the development environment:
 - Project structure, branches, possibly simple CI.

Deliverables:

- Fully functional authentication and role management module.
- Menu structure working with categories and product details.

Week 8 – Nov 24, 2025 to Nov 30, 2025

Focus: Product CRUD & first tests

Activities:

- Implement **RF2.3** – Admin can **add, edit, delete** products.
- Add form validations (supporting RNF1 – quality and robustness).
- Run basic functional tests for RF1 and RF2.
- Perform **Sprint Review** and **Sprint Retrospective** (end of Sprint 1).

Deliverables:

- Complete product management module (CRUD).
- Initial testing report for RF1 and RF2.
- Adjusted backlog for Sprint 2.

Sprint 2 – Cart & Reservations (Weeks 9–10)

Week 9 – Dec 1, 2025 to Dec 7, 2025

Focus: Shopping cart and partial checkout

Activities:

- Implement **RF4 – Cart and Checkout (part 1)**:
 - **RF4.1** – Add, modify, and remove items in the cart.
 - **RF4.3** – Show “no items” message when cart is empty.
- Show basic **subtotal** calculation in the cart.
- Link the cart to authenticated users (cart per user/session).

Deliverables:

- Functional cart (add/modify/remove items).
- Cart view with subtotal and empty-cart message.

Week 10 – Dec 8, 2025 to Dec 14, 2025

Focus: Complete reservation system (RF6)

Activities:

- Finish **RF6 – Reservation System**:
 - **RF6.1** – Time slots cannot overlap for the same table.
 - **RF6.2** – Send confirmation email after successful reservation.
 - **RF6.3** – Admin can cancel or reassign reservations from the panel.
- Integrate reservations with authenticated users.
- Functional testing for RF4 (current state) and RF6.
- **Sprint Review** and **Retrospective** (end of Sprint 2).

Deliverables:

- Fully functional reservation module.
- Test report for reservations and cart.
- Updated backlog with next priorities.

Sprint 3 – Customization, Allergies & Payments (Weeks 11–12)

Week 11 – Dec 15, 2025 to Dec 21, 2025

Focus: Meal customization and allergy management (RF3)

Activities:

- Implement **RF3 – Customization and Allergy Preferences**:
 - **RF3.1** – Link ingredients to each product.
 - **RF3.2** – Allow users to save allergy preferences in their profile.
 - **RF3.3** – Display warnings for allergenic ingredients.
- Make price recalculation dynamic based on added/removed ingredients.
- Run usability tests on the customization process.

Deliverables:

- Working customization feature for meals.
- Allergy management integrated into user profiles.

Week 12 – Dec 22, 2025 to Dec 28, 2025

Focus: Online payment integration (RF5)

Activities:

- Integrate **RF5 – Online Payment Gateway** (Stripe/MercadoPago):
 - **RF5.1** – Process payments over HTTPS, encrypted.
 - **RF5.2** – Failed transactions revert order status to “Pending”.
 - **RF5.3** – Generate a digital receipt automatically.
- Connect payment flow with:
 - Cart (RF4).
 - Reservations/orders.
- Conduct **Sprint Review** and **Retrospective** (end of Sprint 3).

Deliverables:

- Functional payment flow (sandbox mode is fine).
- Orders with statuses: “Pending”, “Approved”.
- Basic digital receipt.

Sprint 4 – Reviews, Promotions & Dashboard (Weeks 13–14)

Week 13 – Dec 29, 2025 to Jan 4, 2026

Focus: Reviews, ratings and loyalty program

Activities:

- Implement **RF7 – Customer Reviews and Ratings:**
 - **RF7.1** – One rating per client per product.
 - **RF7.2** – Average rating displayed under each product.
 - **RF7.3** – Admin can moderate/approve/delete reviews.
- Implement **RF8 – Promotions and Loyalty Program:**
 - **RF8.1** – Validate coupons before payment.
 - **RF8.2** – Loyalty points accumulate after each purchase.
 - **RF8.3** – Admin can create and manage promotional offers.
- Adjust UI to clearly show reviews, ratings, and promotions.

Deliverables:

- Active reviews and ratings module.
- Basic coupons and loyalty points system.

Week 14 – Jan 5, 2026 to Jan 11, 2026

Focus: Administrative dashboard and analytics (RF9)

Activities:

- Complete **RF9 – Administrative Dashboard:**
 - **RF9.1** – Show sales by day, week, and month.

- **RF9.2** – Display low-stock alerts automatically.
- **RF9.3** – Allow export of reports to CSV or PDF.
- Refine admin panel UI/UX.
- **Sprint Review** and **Retrospective** (end of Sprint 4).

Deliverables:

- Fully working admin dashboard with analytics.
- Exportable reports (CSV/PDF).
- End-to-end view of the system.

Phase 4 – Final Testing, Optimization & Documentation (Weeks 15–16)

Week 15 – Jan 12, 2026 to Jan 18, 2026

Focus: Global testing and non-functional requirements (RNF1)

Activities:

- **Unit testing** for key modules (auth, menu, cart, reservations, payment).
- **Integration testing** of full flow:
 - Login → menu → customization → cart → payment → reservation → dashboard.
- **Performance testing** (basic response times and load).
- Strengthen **RNF1 – Performance, Security, and Availability**:
 - Session management, input validation, protection against injections, etc.
 - Error handling and logging.
- Fix critical bugs found during tests.

Deliverables:

- Global testing report (unit, integration, performance, security).
- Bug list and fixes.
- Stable version in a test environment.

Week 16 – Jan 19, 2026 to Jan 25, 2026

Focus: Documentation, polishing & final presentation

Activities:

- Prepare **technical documentation**:
 - Architecture, models, API endpoints, database schema.
 - Deployment/installation guide.
- Prepare **user documentation**:
 - User manual for Clients.
 - User manual for Administrators.
- Build the **final presentation**:
 - Slides and a guided live demo (main use cases).
- Rehearse the presentation and demo (time management, scenarios).

Deliverables:

- Complete technical and user documentation.
- Final presentation (slides) and working demo.
- Final version of the system ready for submission.

6 Test plan